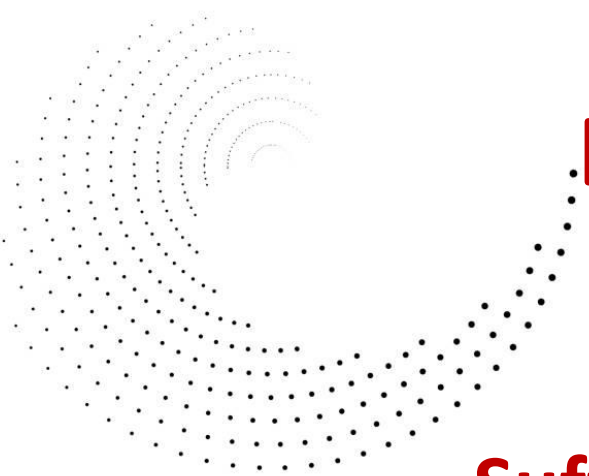




Data Structures and Algorithms -3 (25CS2103E)

SuffixSearch: Text Indexing and Substring Search using Suffix Array

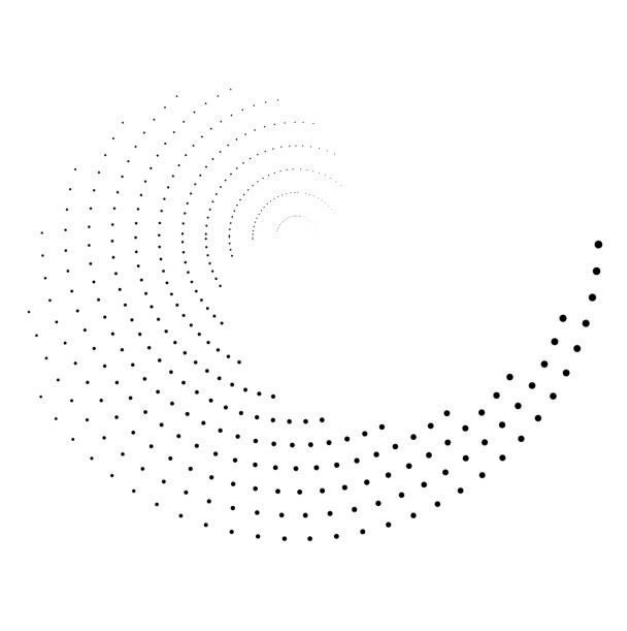
Team Members-1

Name:Sura Rahul Reddy

Rollno:2520030305

Under the Esteemed Guidance of:Dr.

**Name of the DSA-3 faculty name with
designation and department**



Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope



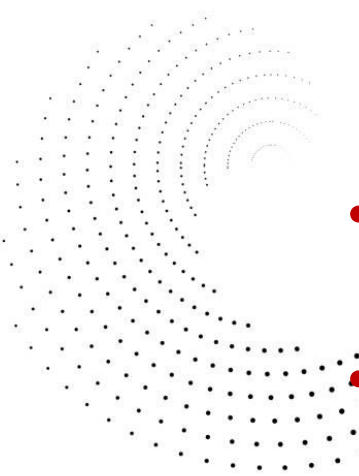
Abstract

This project presents SuffixSearch, an efficient text indexing and substring search tool based on the Suffix Array data structure. The system preprocesses a given text by generating and sorting all its suffixes to construct a suffix array. Once the index is created, users can perform multiple substring search queries efficiently without rebuilding the index. The application displays the generated suffix array and provides search results for user-defined patterns. Binary search is used over the suffix array to reduce the time required for locating matching substrings. The project demonstrates how suffix arrays can be used for efficient text indexing and retrieval, particularly for large text datasets. Performance analysis is included by measuring preprocessing time, search time, and memory usage for different input sizes. The proposed tool provides a practical demonstration of preprocessing-based text searching and highlights the efficiency of suffix arrays compared with repeated brute-force substring searching.



Introduction

- Searching for a substring in large text is a core task in editors, databases, search tools and bioinformatics.
- Brute-force search scans the whole text for every query, costing $O(n \cdot m)$ per pattern.
- A suffix array stores the starting positions of all suffixes of the text in sorted order.
- Because suffixes are sorted, any pattern can be located with binary search in $O(m \log n)$.
- SuffixSearch builds this index once and reuses it for many queries, trading preprocessing time and memory for fast searching.



Literature Survey

- **Weiner (1973)**: introduced the suffix tree, the first linear-time index for substring search.
- **Manber & Myers (1990/93)**: proposed suffix arrays as a compact alternative to suffix trees, searched by binary search.
- **Kasai et al. (2001)**: linear-time construction of the LCP array, which speeds up suffix array searches.
- **Kärkkäinen & Sanders (2003)**: the DC3 (skew) algorithm builds suffix arrays in linear time.
- **Nong, Zhang & Chan (2009)**: SA-IS, a fast linear-time induced-sorting construction used in practice.



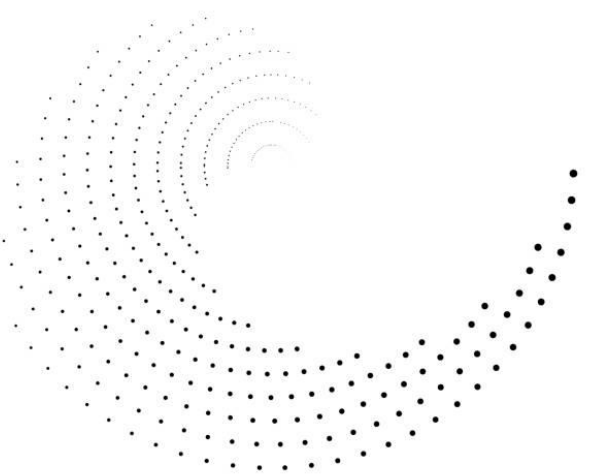
Problem Statement and Objectives

Problem Statement

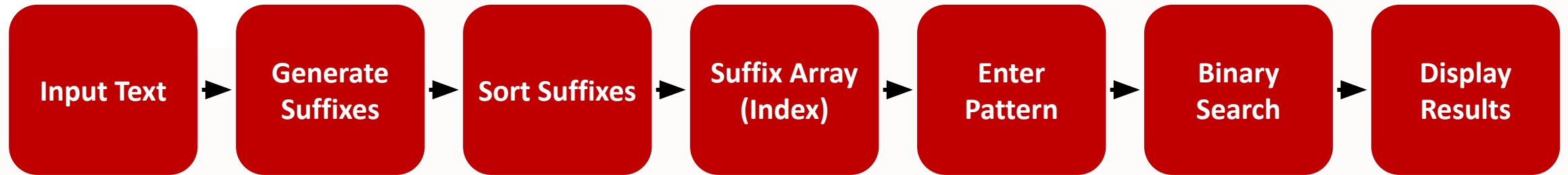
- Repeated brute-force substring searching over large text is slow, since every query rescans the full text.

Objectives

- Build a suffix array by generating and sorting all suffixes of the input text.
- Answer multiple substring queries using binary search, without rebuilding the index.
- Display the suffix array and the search results for user-defined patterns.
- Measure preprocessing time, search time and memory usage for different input sizes.
- Compare suffix array search with brute-force search.



WORKFLOW

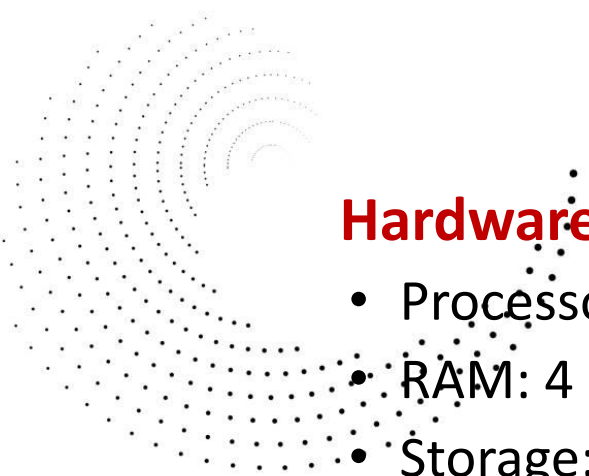


Steps 1–4 run once (preprocessing). Steps 5–7 repeat for every query using the same index.

Results

Text size (chars)	Preprocessing (ms)	Suffix array search, 100 queries (ms)	Brute-force search, 100 queries (ms)	Index memory (MB)
1,000	0.7	0.07	0.1	0.004
10,000	16.3	0.11	1.3	0.04
50,000	72.4	0.20	13.8	0.20
100,000	95.3	0.32	24.6	0.40

- Java (JDK 21), random lowercase text, patterns of length 5. Both methods returned identical match counts. Index memory = int[] suffix array (4 bytes per suffix).
- Suffix array search stays well under 1 ms per 100 queries, while brute force grows linearly with text size.



Hardware and Software requirements

Hardware

- Processor: Intel Core i3 or higher (or equivalent)
- RAM: 4 GB minimum (8 GB recommended for large texts)
- Storage: 500 MB free disk space

Software

- Operating system: Windows 10/11, Linux or macOS
- Language: Java (JDK 11 or higher)
- Tools: IntelliJ IDEA / Eclipse / VS Code



Conclusion

- SuffixSearch builds a suffix array once and answers many substring queries without rebuilding the index.
- Binary search over the sorted suffixes finds matches in $O(m \log n)$ instead of scanning the whole text.
- In our tests, searching 100 patterns in a 100,000-character text took about 0.3 ms, compared with about 25 ms for brute force.
- The cost is preprocessing time and extra memory for the index, which pays off when many queries are made on the same text.



Future Scope

- Use linear-time construction algorithms such as DC3 or SA-IS to reduce preprocessing time.
- Add an LCP array to speed up searching and support repeated-substring queries.
- Support very large files and genome-scale datasets with memory-efficient or disk-based indexes.
- Add case-insensitive, multiple-pattern and approximate matching, plus a graphical interface.

CODE-SCREENSHOTS

Suffix Array logic:

```
for (int gap = 1; gap < n; gap <= 1) {
    final int g = gap;
    final int[] rk = rank.clone();

    java.util.Arrays.sort(sa, (a, b) -> {
        if (rk[a] != rk[b]) return Integer.compare(rk[a], rk[b]);
        int ra = (a + g < n) ? rk[a + g] : -1;
        int rb = (b + g < n) ? rk[b + g] : -1;
        return Integer.compare(ra, rb);
    });

    tmp[sa[0]] = 0;
    for (int i = 1; i < n; i++) {
        tmp[sa[i]] = tmp[sa[i - 1]];
        boolean same = (rk[sa[i]] == rk[sa[i - 1]]);
        if (same) {
            int ai = sa[i] + g < n ? rk[sa[i] + g] : -1;
            int bi = sa[i - 1] + g < n ? rk[sa[i - 1] + g] : -1;
            same = (ai == bi);
        }
        if (!same) tmp[sa[i]]++;
    }
    System.arraycopy(tmp, 0, rank, 0, n);
    if (rank[sa[n - 1]] == n - 1) break;
}
```

Kasai's O(n) LCP Algorithm:

```
// Kasai O(n) LCP
private int[] buildLCPArray() {
    int[] lcp = new int[n];
    int h = 0;
    for (int i = 0; i < n; i++) {
        if (rank[i] > 0) {
            int j = sa[rank[i] - 1];
            while (i + h < n && j + h < n && text.charAt(i + h) == text.charAt(j + h)) h++;
            lcp[rank[i]] = h;
            if (h > 0) h--;
        }
    }
    return lcp;
}
```

FRONTEND-SCREENSHOT

SUFFIX SEARCH

TEXT TO INDEX

Enter text here...

SEARCH PATTERN

Enter pattern to search...

EXECUTE SEARCH

System ready. Awaiting input...

SAMPLE I/O EXAMPLES

Basic Pattern Matching:

No Pattern Available:

TEXT TO INDEX

HYDERABAD

SEARCH PATTERN

HY

EXECUTE SEARCH

STATUS: MATCH FOUND
COUNT: 1
POSITIONS: [0]
TIME: 10.200 μ s

TEXT TO INDEX

mississippi

SEARCH PATTERN

miss2

EXECUTE SEARCH

STATUS: NOT FOUND
TIME: 5.100 μ s

Edge Case:

TEXT TO INDEX

abababab

SEARCH PATTERN

abab

EXECUTE SEARCH

STATUS: MATCH FOUND

COUNT: 3

POSITIONS: [0, 2, 4]

TIME: 12.100 μ s

TEXT TO INDEX

ATCGATCGATCGATCGATCGATCGATCGATCGATCGATCGATCGATCGATCG
ATCGATCGATCGATCGATCGATCGATCGATCGATCGATCGATCGATCGATCG

SEARCH PATTERN

ATCG

EXECUTE SEARCH

STATUS: MATCH FOUND

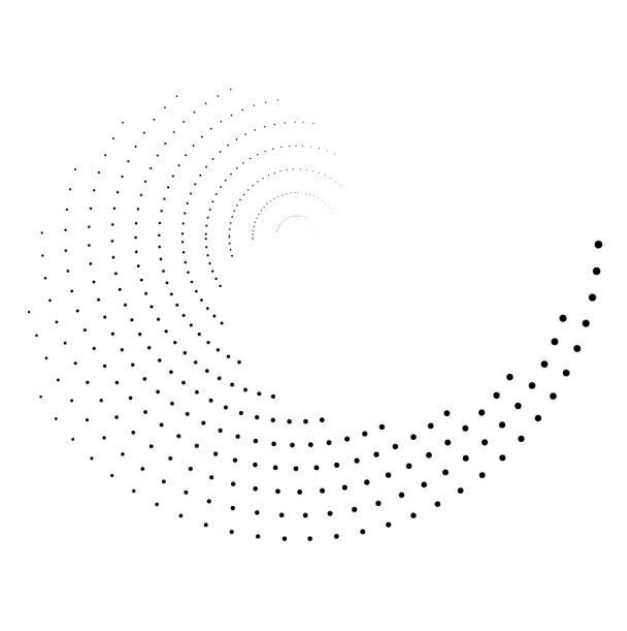
COUNT: 25

POSITIONS: [0, 4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48,
52, 56, 60, 64, 68, 72, 76, 80, 84, 88, 92, 96]

TIME: 7.600 µs

Geo-Tag Photo





Thank you